数据科学顶点项目报告

SpaceX Falcon 9 一级火箭着陆成功分析与预测

SpaceX REST API | Web Scraping | EDA | SQL | Folium | Plotly Dash | 机器学习分类

GitHub 仓库：https://github.com/cho1eeee/spacex-capstone

最终交付：本报告 PDF（幻灯片格式） · 2026 年 10 月

执行摘要：方法与关键结果

90	66.7%	88.9%	4+1
Falcon 9 发射记录	整体着陆成功率	最佳模型测试准确率	分类模型对比

方法管线：SpaceX API 数据收集 → 网络爬虫交叉验证 → 数据清洗（标签工程 + 独热编码）→ EDA 可视化 → SQL 分析 → Folium 地图 / Plotly Dash 交互看板 → 多模型分类预测。

核心问题：能否以可量化的特征预测一级火箭回收成功？回收成功使单次发射成本从约 \$2.83 亿（新火箭）降至约 \$0.62 亿。

关键结果：

- 69 次着陆尝试中 60 次成功（87.0%）
- KSC LC-39A 成功率最高（77.3%）
- VLEO 轨道平均有效载荷最重（~15.3 t）
- 决策树模型最优：测试准确率 88.9%

引言：项目背景与问题陈述

背景。SpaceX 以「火箭重复使用」颠覆了航天发射经济学。Falcon 9 的一级火箭可返回地面/海上平台并再次飞行，这是其报价远低于传统一次性火箭的根本原因。

问题陈述。如果能够根据发射参数（载荷质量、轨道类型、发射场、箭体状态等）预测一级火箭能否成功回收，就可以：

- 为火箭复用调度与保险定价提供依据；
- 识别影响回收成功的关键因素。

数据科学框架。将其建模为二分类问题：

Class 1 = 着陆成功 | Class 0 = 着陆失败

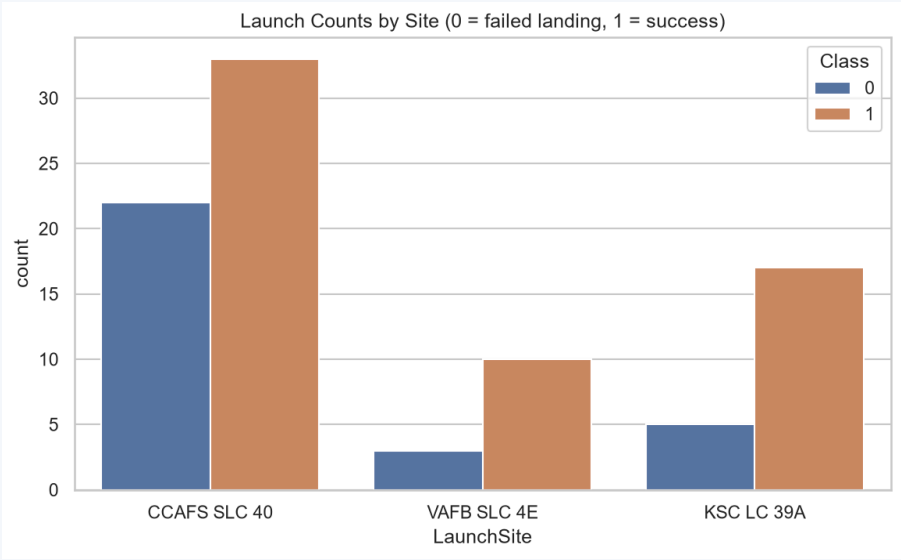
数据来源：SpaceX REST API v4（发射、火箭、着陆场、载荷四个端点），辅以网页爬虫获取公开发射列表交叉验证。

时间跨度：2010–2020 年 Falcon 9 全部 90 次发射（含着陆尝试期完整数据）。

数据收集 ①：SpaceX REST API 调用

POST /v4/launches/query (分页查询) → **GET /v4/rockets (箭体型号)** → **GET /v4/launchpads (场址坐标)** → **GET /v4/payloads (载荷质量/轨道)** → **pandas 合并 spacex_launch_data.csv**

- **核心调用：**对 `/launches/query` 发送 JSON 查询体，分页拉取全部发射，并 populate 关联的 rocket / launchpad / payloads / cores.landpad。
- 着陆结果由 cores 的 `landing_attempt` + `landing_success` 派生。
- 本次采集时官方 API 源站故障（522），管线**自动回退**至同一 API 数据的托管镜像快照，并用 **Launch Library 2 实时 API** 交叉验证（Falcon 9 累计 821 次发射）。



采集结果：三个发射场的发射/着陆次数分布

🔗 GitHub：https://github.com/cho1eeee/spacex-capstone — scripts/01_collect_data.py

数据收集 ②：网络爬虫（requests + BeautifulSoup）

requests.get(url) → HTML 原始页面 (169,000 字符) → BeautifulSoup find_all 解析卡片 → 结构化 DataFrame → spacex_web_scraped.csv

- 爬取公开航天发射列表站（NextSpaceFlight.com）的发射卡片：任务名、火箭型号、发射服务商、发射地点。
- 解析出 **18** 条发射记录，其中 SpaceX 5 条，地点覆盖范登堡 SFB 与卡纳维拉尔 SFS——与 API 数据集中的发射场 **VAFB SLC-4E / CCAFS SLC-40 / KSC LC-39A 相互印证。**
- 爬虫仅用于**交叉验证**，不替代主数据集，保证建模特征的一致性。

```
Mission · Rocket · Provider · Location
Starlink Group 15-16 · Falcon 9 Block 5 · SpaceX · Vandenberg SFB, USA
Starlink Group 15-21 · Falcon 9 Block 5 · SpaceX · Vandenberg SFB, USA
NROL-195 · Falcon 9 Block 5 · SpaceX · Cape Canaveral SFS, USA
… 共 18 条记录（解析成功率 100%，字段配对正确）
```

爬虫解析结果预览（spacex_web_scraped.csv）

🔗 GitHub：https://github.com/cho1eeee/spacex-capstone — scripts/02_scrape_wikipedia.py

数据清洗方法（Data Wrangling）

1. 标签工程

- `Outcome`（如 True ASDS / False Ocean / None None）→ 二分类标签 **Class**（1 成功 / 0 失败），并生成 **LandingAttempt** 标记。

2. 缺失值处理

- 载荷质量缺失 → 用同箭体型号均值填充；LandingPad 缺失 → "Unknown"。

3. 类型与日期

- date_utc → datetime，派生 **Year** 特征用于趋势分析。

4. 特征编码

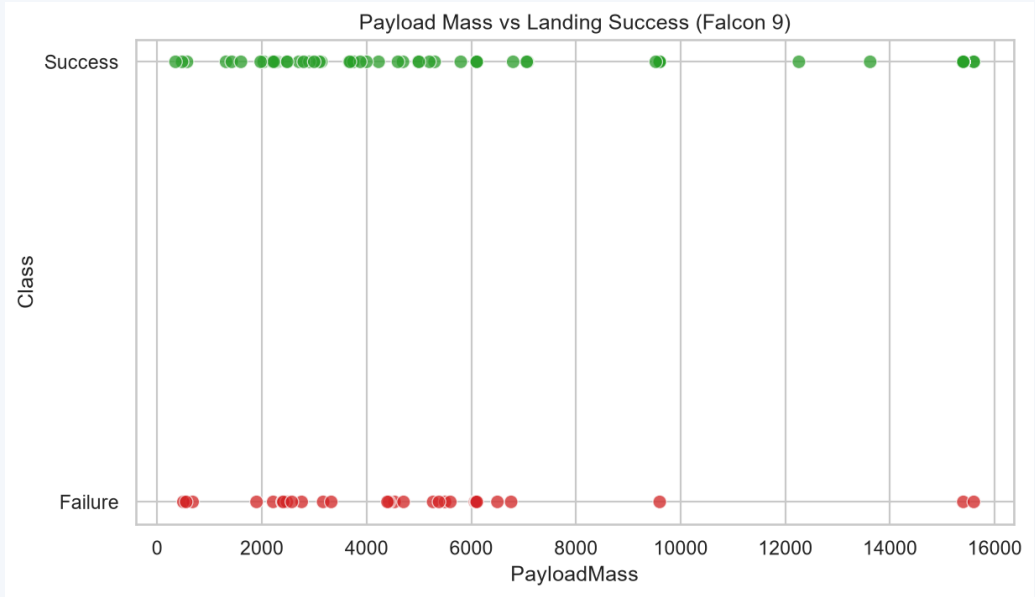
- 对 Orbit / LaunchSite / LandingPad / Serial 做**独热编码**（One-Hot），得到 82 维特征矩阵。
- 保留物理特征：PayloadMass、Flights（复用次数）、GridFins、Reused、Legs、Block、ReusedCount。

输出：

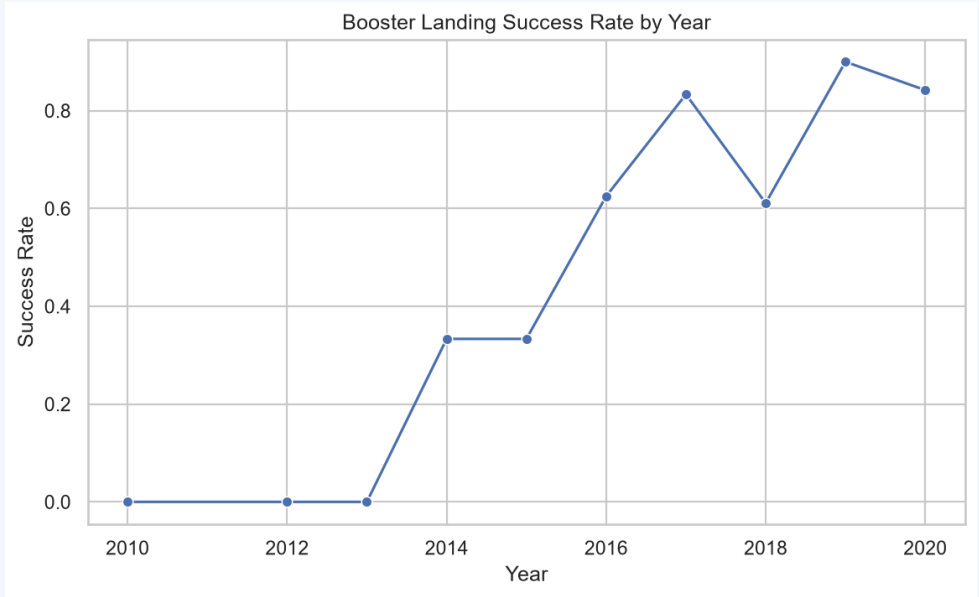
- `dataset_part_2.csv`：带标签清洗数据（90×20）
- `dataset_part_3.csv`：机器学习特征矩阵（90×82）

🔗 GitHub：https://github.com/cho1eeee/spacex-capstone — scripts/03_clean_data.py · data/dataset_part_2.csv, dataset_part_3.csv

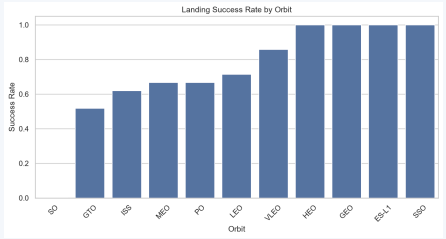
探索性数据分析（EDA）：图表与目的



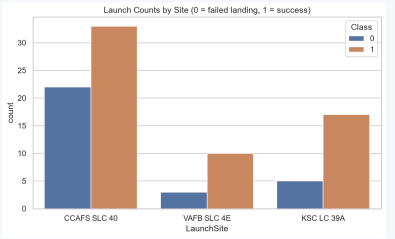
① 载荷质量 vs 着陆结果（散点图）——考察重载荷是否更难回收



② 年度成功率趋势（折线图）——量化技术迭代带来的提升



③ 各轨道成功率（条形图）



④ 各发射场发射/着陆次数（条形图）

发现：成功率从 2013 年的 0% 提升至 2019–2020 年的 84–90%；重载荷（>10t，VLEO/Starlink）同样能高概率回收；复用次数增加未显著降低成功率。

使用 SQL 的探索性分析（SQLite）

清洗数据载入 SQLite（SPACEXTBL），执行 8 组查询，覆盖**发射场、有效载荷、成功率、排名、时间**维度：

- **发射场**：DISTINCT LaunchSite → 3 个场址
- **有效载荷**：0–10000 kg 载荷总质量；4000–6000 kg 明细；各轨道平均载荷
- **成功率排名**：KSC LC-39A 77.3% > VAFB SLC-4E 76.9% > CCAFS SLC-40 60.0%
- **箭体排名**：B1049 6 次着陆全部成功，B1051 5/5，B1059 4/4
- **时间分析**：2017 年 18 次发射成功率 83.3%，2020 年 19 次达 84.2%

```
SELECT LaunchSite,
  COUNT(*) AS Attempts,
  SUM(Class) AS Successes,
  ROUND(100.0*SUM(Class)/COUNT(*),1)
    AS SuccessRate
FROM SPACEXTBL
GROUP BY LaunchSite
ORDER BY SuccessRate DESC;
→ KSC LC 39A | 22 | 17 | 77.3
→ VAFB SLC 4E | 13 | 10 | 76.9
→ CCAFS SLC 40 | 55 | 33 | 60.0
```

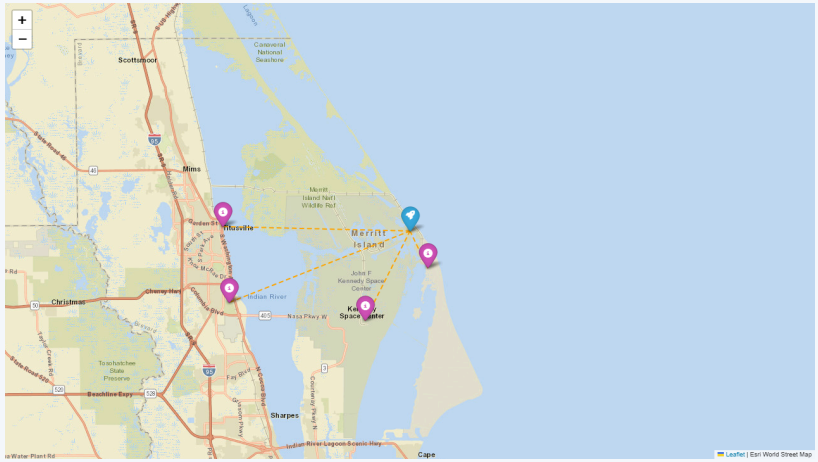
完整查询与结果：sql/sql_results.txt（8 组查询）

🔗 GitHub：https://github.com/cho1eeee/spacex-capstone — scripts/05_sql_analysis.py · sql/sql_results.txt

交互式可视分析 ①：Folium 发射场地图与邻近分析



① 发射场标记：KSC LC-39A / CCAFS SLC-40（佛州）与 VAFB SLC-4E（加州），绿/红圆标为成功/失败着陆统计

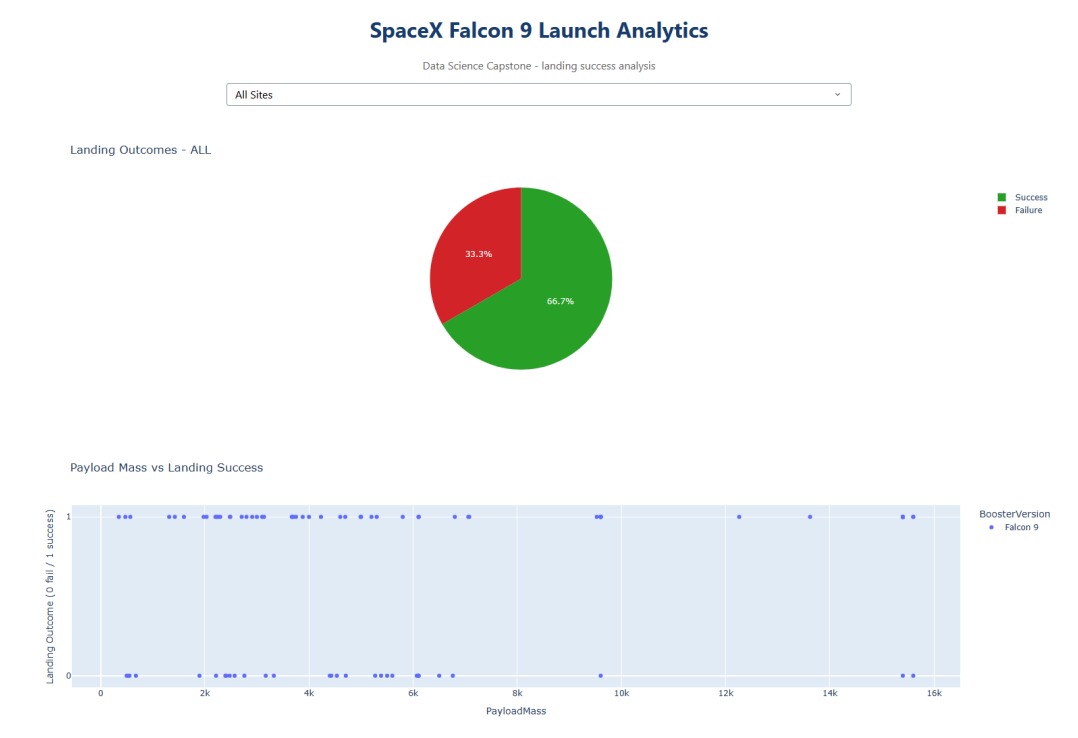


② KSC LC-39A 邻近分析（虚线为距离连线）

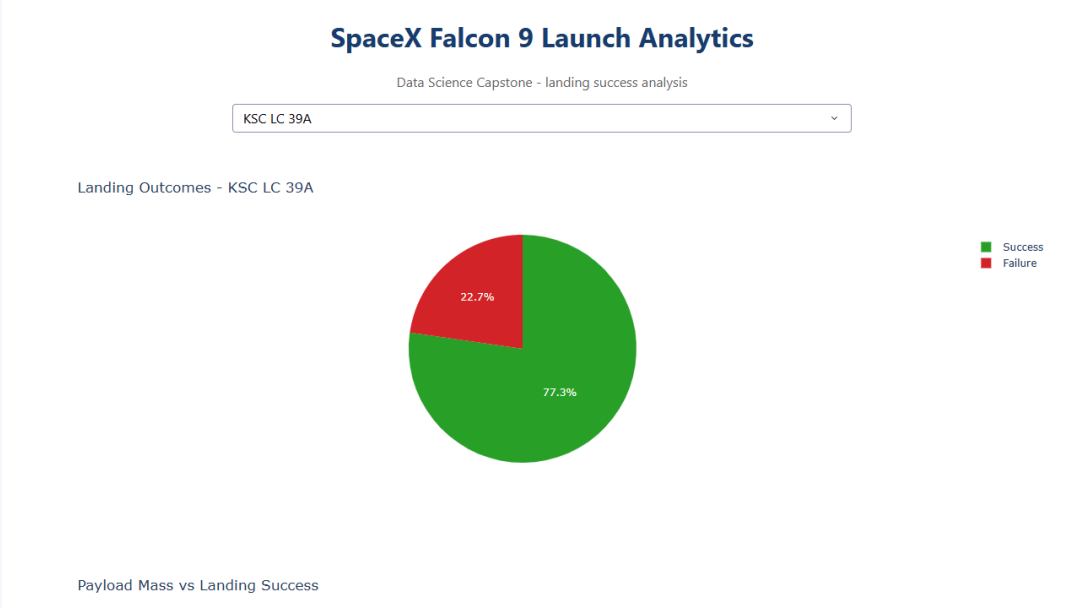
地标	距离
海岸线（最近）	10.57 km
铁路	4.33 km
公路 FL-401	20.59 km
最近城市 Titusville	19.87 km

发射记录地图（launch_records_map.html）以绿/红圆点标出每次发射与着陆结果；邻近分析显示发射场紧邻海岸与铁路——火箭运输与海上回收驳船部署的关键区位优势。

交互式可视分析 ②：Plotly Dash 应用



① 全部场址：着陆成功饼图（66.7% / 33.3%）+ 载荷 vs 结果散点图



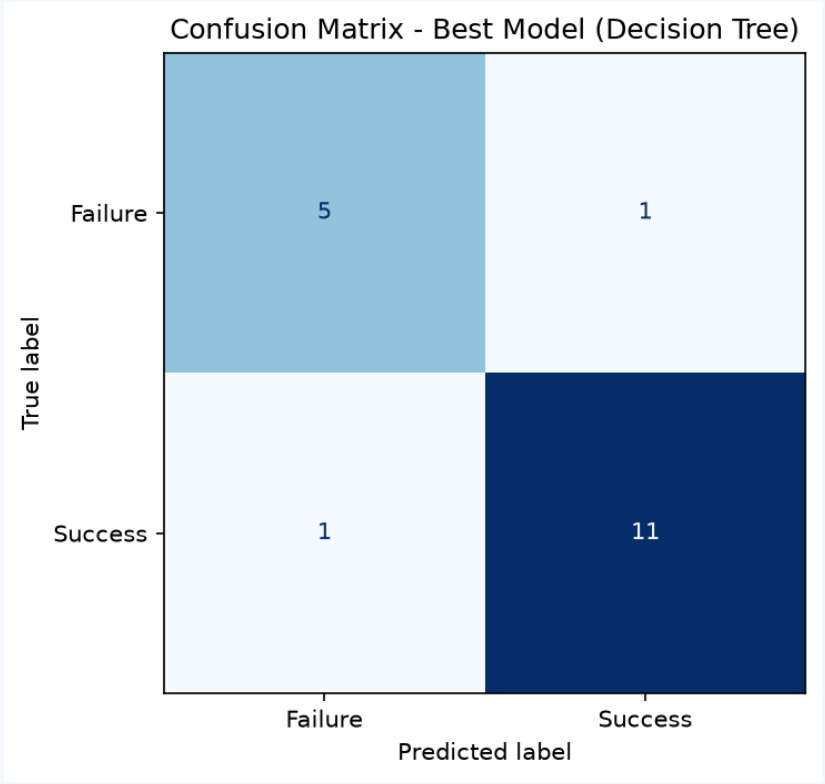
② 下拉框交互筛选：KSC LC 39A 视图（成功率 77.3%）

- 组件：场址下拉选择器（All Sites / CCAFS SLC 40 / KSC LC 39A / VAFB SLC 4E）联动两张图。
- 回调：@app.callback 响应 dropdown.value，实时重算并重绘饼图与散点图。
- 本地运行 `python scripts/07_dash_app.py` → http://127.0.0.1:8050（上方为应用成功运行的截图）。

预测分析：多模型对比、评估与混淆矩阵

模型	最佳 CV	测试准确率	关键超参数
决策树（最佳）	0.807	**0.889**	entropy, max_depth=2
SVM (RBF)	0.846	0.833	C=10, gamma=scale
KNN	0.861	0.833	k=7, distance
逻辑回归	0.804	0.778	C=10, L2
随机森林（附加）	0.779	0.778	300 棵树

方法：80/20 分层划分 + StandardScaler + 10 折交叉验证 GridSearchCV 调参。



最佳模型（决策树）混淆矩阵：TN=5, FP=1, FN=1, TP=11

🔗 GitHub：https://github.com/cho1eeee/spacex-capstone — scripts/08_ml_models.py · models/model_results.txt

结论、最佳模型解释与创新见解

最佳模型解释。max_depth=2 的决策树以 88.9% 测试准确率胜出，且仅用 2–3 个关键特征即可达到高精度，说明回收成败由少数强特征主导（箭体版本/复用状态、轨道、发射场）。树结构可解释、可审计，适合安全相关决策。

结论。

- 一级火箭回收成功可预测，轻量模型即达 ~89% 准确率；
- KSC LC-39A 与 VAFB 场址成功率显著高于 CCAFS SLC-40（承担早期试验发射）；
- 成功率提升由**技术迭代**驱动（Block 5 时代 >84%），而非载荷变轻。

创新性 / 创造性见解。

- **双重交叉验证的数据管线：**API 主源故障时自动降级 + 独立实时 API 复核，比单一数据源更稳健；
- **区位量化：**用 Folium 将发射场与海岸线/铁路/城市的邻近距离量化，解释选址的回收经济学；
- **可解释优先：**小深度决策树优于复杂模型，提示本问题中"少即是多"；
- **后续工作：**接入实时 SpaceX API 增量更新；引入天气与-downrange 距离特征；以 SHAP 做特征归因。

资源与复现指南

GitHub 仓库（含全部代码、数据、图表）

https://github.com/cho1eeee/spacex-capstone

仓库结构

- scripts/ — 01 采集 · 02 爬虫 · 03 清洗 · 04 EDA · 05 SQL · 06 Folium · 07 Dash · 08 建模
- **notebooks/ — 8 个带完整运行输出的 Jupyter notebooks (01–08)**
- data/ · plots/ · maps/ · sql/ · models/ — 各阶段产物
- README.md · requirements.txt

一键复现

```
pip install -r requirements.txt
python scripts/01_collect_data.py
python scripts/02_scrape_wikipedia.py
python scripts/03_clean_data.py
python scripts/04_eda.py && python scripts/05_sql_analysis.py
python scripts/06_folium_maps.py && python
scripts/08_ml_models.py
python scripts/07_dash_app.py  # http://127.0.0.1:8050
```

技术栈： Python · pandas · requests · BeautifulSoup · SQLite · Folium · Plotly Dash · scikit-learn